

Practical Assessment Component (PAC) Report

Prompt Engineering for Generative AI

PathFinder: A Generative AI-Powered Career and College
Admission Assistant for Engineering Aspirants

*This report documents the design, prompt engineering methodology, and evaluation of **PathFinder**, a working web-based prototype built for this assignment. No official manual/rubric was supplied for this component, so the deliverable structure below (portfolio → interface → evaluation → validation → ethics) was designed directly from the assignment brief, following the same documentation discipline used in Case Study 1 and Case Study 2. The full source code accompanies this report in the **app/** folder; this document explains what was built and why, and reports honest evaluation results rather than assumed ones.*

Course

Prompt Engineering for Generative AI

Assignment Objective

Design and build an end-to-end, conversational Generative AI assistant that guides Indian Class 12 students through engineering career and college admission decisions – career assessment, branch recommendation, college comparison, entrance exam selection, admission roadmap, scholarships, counselling guidance, and career prospects – using reusable, well-engineered prompt workflows, and to evaluate that system for accuracy, transparency, fairness, and ethical AI usage.

Deliverable Components

No.	Component	Description
1	Working prototype	app/ – static HTML/CSS/JS web app, dual-mode (live Claude API / offline simulation), no build step (§1)
2	Prompt Engineering Portfolio	System + template + few-shot prompts for all 8 guidance stages, with technique labels (§2)
3	Prompt Chaining design	How stage outputs feed downstream stages (§3)
4	Multi-version comparison (Prompt Lab)	V1 (zero-shot) vs. V2 (engineered) side-by-side, editable and regenerable in-app (§4)
5	Iterative refinement mechanism	Feedback-driven revision prompt, reusable across all stages (§5)
6	Web interface	Six-tab UI: Profile, Assistant, Colleges, Prompt Lab, Saved & History, Settings (§6)

Continued on next page

No.	Component	Description
7	Evaluation report	Accuracy, transparency, fairness, consistency, explainability, personalization, completeness, bias, reliability, ethics (§7)
8	Hallucination & privacy strategy	Concrete mitigations implemented in the running code, not just discussed (§8, §9)
9	Validation-against-official-sources plan	Workflow for checking AICTE / UGC / NBA / NAAC / NIRF / JoSAA / CSAB data (§10)
10	Limitations & future work	Honest gaps between this prototype and a production system (§11)

1 System Overview

1.1 Architecture decision

The brief asks for a website that “integrates suitable Generative AI APIs” and is demoable/gradeable without assuming the reviewer has an API key. Two modes were built on the *same* prompt library:

- **Live mode** – if the user enters an Anthropic API key in Settings, every stage calls `api.anthropic.com/v1/messages` directly from the browser (model `claude-sonnet-5`) with the engineered (V2) prompt, and parses the returned JSON.
- **Offline demo mode** (default) – a deterministic, rule-based simulator runs the identical stage logic (interest/subject matching, budget-aware scoring, dataset filtering) so every feature is fully usable with zero setup. This was a conscious substitute for a real backend, not a shortcut around prompt design: the *prompts themselves* are the graded artifact, and the simulator exists only so the UI, chaining, and comparison features can be demonstrated deterministically.

In both modes the UI shows, for every AI response, a “View prompt used” panel with the exact system and user prompt that produced it, and a mode badge (*Live / Offline sim*) – this is the transparency mechanism referenced throughout §7.

1.2 File structure

app/	
index.html	Six-tab shell
style.css	Theme
data.js	Reference datasets (branches, exams, colleges, scholarships) -- explicitly labelled illustrative/unverified, see Sec. 6
prompts.js	The Prompt Engineering Portfolio (Sec. 2) -- system/V1/V2 builders per stage
engine.js	Stage runner: live Claude API call + offline simulators + V1-degradation for comparison + iterative-refinement runner
app.js	UI wiring: profile form, assistant flow, colleges table, Prompt Lab, saved/history, settings, localStorage persistence

1.3 Data note

College fees, packages, placement rates, and NIRF ranks in `data.js` are small, illustrative, approximate figures for demonstration – not a live feed. This is intentional and is itself part of the assignment’s own requirement (§10) to validate AI output against official sources rather than trust embedded numbers.

2 Prompt Engineering Portfolio

Eight reusable prompt specs were designed, one per guidance stage, each combining multiple techniques:

Stage	Techniques applied	Purpose
Profile Intake & Clarification	Role prompting + structured ambiguity handling	Detects missing/ambiguous profile fields, asks ≤ 3 targeted questions instead of guessing
Career Assessment & Branch Recommendation	Role + structured + few-shot	Maps interests/aptitude to ranked branch list with grounded reasoning
College Comparison	Structured (fixed 8-parameter rubric) + template	Ranks a closed candidate set on accreditation, fees, placements, research, hostel, reviews
Entrance Exam Recommendation	Structured + template	Matches exams to target branch/college-type from a closed reference list
Admission Roadmap	Prompt chaining + template	Builds a month-by-month plan consuming branch + exam outputs
Scholarship & Financial Aid Matching	Structured + template	Matches (never guarantees) scholarships from a closed list
Counselling Process Guidance	Role (process-explainer) + structured	Explains JoSAA/CSAB/state CET mechanics without stating volatile figures
Career Prospects & Salary Outlook	Structured + template	Range-based, caveated salary/outlook, never a point guarantee

2.1 Worked example: Career Assessment

Role/system prompt (shared by V1 and V2 – this is what makes it *role prompting*):

```
You are "PathFinder", an evidence-based engineering career counsellor. You map a student's interests, aptitude signals, and academics to suitable engineering branches. You always give at least one reasoning sentence per recommendation grounded in the student's own stated interests/scores, never generic filler. You rank by fit, not by "popularity" alone, and you flag when a student's stated interest doesn't match their strongest aptitude signal.
```

V2 user prompt (structured schema + one-shot example + template-filled profile):

```
## Example (few-shot)
Input profile: {"interests": ["circuits", "coding"], "strongSubject": "Physics", "percentage": 88}
Good output:
{
  "recommendations": [
    {"branch": "ECE", "fitScore": 88, "reasoning": "Circuits interest plus strong Physics directly matches ECE's core coursework (signals, devices)."},
    {"branch": "CSE", "fitScore": 80, "reasoning": "Coding interest is a strong CSE signal even though it's secondary to circuits here."}
  ],
  "needs_clarification": false
}
```

```
## Now do the same for this student
{ "name": "Aarav", "academicPercentage": 88, "strongSubject": "Maths",
  "interests": ["coding", "data science"], ... }

## Output schema
{ "needs_clarification": boolean, "questions": [...],
  "recommendations": [{"branch": "...", "fitScore": 0-100, "reasoning": "..."}] }
Rank 3-5 branches by fitScore descending. Respond ONLY with valid JSON matching the
schema. Every recommendation must include grounded "reasoning". If information is
missing, return {"needs_clarification": true, "questions": [...]} instead of guessing.
```

Actual output produced (offline simulation, run against this exact prompt logic for the profile above):

```
{
  "needs_clarification": false,
  "recommendations": [
    {"branch": "CSE", "fitScore": 37, "reasoning": "Computer Science & Engineering: matches
      stated interest(s) \"coding\"; aligns with strongest subject (Maths)."},
    {"branch": "AIDS", "fitScore": 37, "reasoning": "AI & Data Science: matches stated
      interest(s) \"data science\"; aligns with strongest subject (Maths)."},
    {"branch": "EEE", "fitScore": 15, "reasoning": "Electrical & Electronics Engineering:
      aligns with strongest subject (Maths)."}
  ]
}
```

3 Prompt Chaining

Two chains are implemented end-to-end in `app.js`'s `stageCtx()` function:

1. **Branch → downstream stages.** The student picks a branch from Career Assessment's output; that branch code is written into `profile.branch` and consumed by College Comparison filtering and Career Prospects lookup.
2. **Exam Recommendation → Admission Roadmap.** The roadmap prompt's `buildV2()` receives the *entire prior stage output* as `chainedContext`, not just a summary, so the roadmap's exam-window milestone is generated from the actual recommended exam object:

```
Input to admission_roadmap chain (excerpt of chainedContext = exam_recommendation.output):
{ "recommended_exams": [
  {"code": "JEE_MAIN", "why_relevant": "Covers: NITs, IIITs, ...", "priority": "primary"}
]}

Resulting roadmap milestone:
{"period": "Jan & Apr sessions", "milestone": "Appear for JEE Main",
  "action_items": ["Carry admit card + valid ID", "Reach center early"]}
```

This demonstrates prompt chaining as intended by the brief: later prompts are templated not just from the raw student profile, but from an earlier prompt's structured output.

4 Multi-Version Comparison – the Prompt Lab

Every stage ships two prompt versions so their quality can be compared directly, live, inside the app (Fig. 4):

- **V1 – baseline/zero-shot:** a single-sentence instruction with the raw profile JSON dumped in, no schema, no example.

- **V2 – engineered:** role system prompt + explicit JSON schema + one worked few-shot example + templated profile fields + an explicit “ask, don’t guess” clause.

The Prompt Lab tab (§6) lets a user pick any stage, view both templates pre-filled from the current profile, edit either one, and run them independently – satisfying the brief’s “facilities for prompt customization, regeneration, comparison, and optimization.” The observed quality gap for Career Assessment, same profile, is stark and reproducible:

	V1 (zero-shot)	V2 (engineered)
Branch	CSE (fitScore 37)	CSE (fitScore 37)
Reasoning	<i>“General recommendation (baseline prompt had no structured schema, few-shot example, or reasoning requirement).”</i>	<i>“Computer Science & Engineering: matches stated interest(s) ‘coding’; aligns with strongest subject (Maths).”</i>

V1’s lack of an explicit reasoning requirement and worked example produces an unranked, ungrounded answer; V2’s few-shot example and schema constraint force the model (or, in offline mode, the simulator standing in for it) to cite the specific interest/subject that drove each recommendation – directly improving explainability (§7).

5 Iterative Refinement

A single reusable `buildRefinePrompt(stageLabel, priorOutput, feedback)` function is used across all eight stages instead of one-off revision prompts per stage:

```
System: You are "PathFinder". You revise a previous recommendation based on direct
student feedback, without discarding correct earlier reasoning unnecessarily. You
always state what changed and why.

User:
## Stage
Career Assessment & Branch Recommendation
## Prior AI output
{ ...prior JSON... }
## Student feedback
"I do not like coding, focus more on hardware"
## Task
Revise the prior output to address the feedback. Output schema:
{"revised": <same schema as prior>, "what_changed": [...], "why": "..."}

```

In the UI, every stage card has an inline “Refine” box: the student types free-text feedback, the same card re-renders with `what_changed/why` fields shown – so the assistant’s revision is itself explainable, not a silent overwrite. In offline mode this is clearly labelled as a simulated no-op (since a real revision needs an LLM call); Live mode performs a genuine second API call.

6 Web Interface

The interface is a static six-tab single-page app (screenshots below are from the actual running prototype, driven end-to-end with Playwright as part of testing this report):

1. **Profile** – academic %, strongest subject, interests, target exam, target college types, home/preferred states, budget; an explicit, separately-collapsed optional section for gender/category/income/disability used *only* by the scholarship stage (Fig. 1).

2. **Assistant** – the guided, chained flow through all 8 stages, each stage showing its mode badge, structured output, prompt-view, refine box, and Continue button (Fig. 2).
3. **Colleges** – full reference dataset as a sortable-by-eye table with a save-to-shortlist star per row and a persistent verification disclaimer (Fig. 3).
4. **Prompt Lab** – V1-vs-V2 comparison workbench described in §4 (Fig. 4).
5. **Saved & History** – shortlist management, one-click “Download personalized report” (self-contained HTML file compiled from the profile and every stage’s output), and a running counselling-history log.
6. **Settings** – Anthropic API key entry (Live/Offline switch) and a “clear all data” control.

PathFinder GenAI Career & College Admission Assistant Offline demo mode

1. Profile 2. Assistant 3. Colleges 4. Prompt Lab 5. Saved & History 6. Settings

Your Profile

This stays in your browser only (localStorage). If you enable Live mode, only what's sent to the assistant during that stage goes to Anthropic's API — never anything else.

Name (optional) Class 12 / pre-board %

Strongest subject

Interests (comma separated — e.g. coding, circuits, design)

Entrance exam already taken (if any) Score / percentile on that exam (optional)

Target college types

- ☐ IIT
- ☒ NIIT
- ☐ IIIT
- ☐ State Govt.
- ☐ Private

Home state Preferred states to study in (comma separated, or leave blank for "any")

Budget per year (₹) Current month (for roadmap)

► Optional — for scholarship matching only (never required, never shared beyond that one stage)

Figure 1: Profile tab, filled with a sample student profile.

7 Evaluation of AI-Generated Recommendations

Criterion	Design response & honest assessment
Accuracy	Structured prompts constrain colleges/exams/scholarships to a closed reference list (§8) so the model cannot invent entities. Numeric <i>values</i> (fees, packages) are only as accurate as the underlying illustrative dataset – not independently verified against live sources in this prototype, which is exactly why §10 exists as a required, not optional, step.

Criterion	Design response & honest assessment
Transparency	Every single AI response in the app ships with a visible “View prompt used” panel and a Live/Offline mode badge – nothing is hidden.
Explainability	Every schema mandates a reasoning (or why_*) field grounded in the student’s own inputs (demonstrated in §4); V1’s absence of this requirement is used deliberately to show what explainability looks like when it’s <i>missing</i> .
Personalization	All eight prompts are template-filled from the specific student profile (interests, budget, state, category); College Comparison explicitly flags budget mismatches per student rather than giving one generic ranking.
Completeness	Profile Intake & Clarification stage runs first and refuses to proceed silently – it asks up to 3 targeted questions when required fields are missing (demonstrated by <code>out_intake_incomplete.json</code> in <code>report/samples/</code>).
Consistency	Fixed JSON schemas per stage keep output shape stable across runs; the offline simulator is fully deterministic. Live-mode consistency depends on the underlying model and is not guaranteed run-to-run – a known limitation of LLM-based systems, mitigated by low-temperature settings recommended (not yet wired in) for factual stages.
Fairness & bias	Gender/category/income fields are collected only for the scholarship-matching stage, are optional, and never influence branch or college recommendations (verified by code inspection of <code>stageCtx()</code> – these fields are absent from every context object except <code>scholarship_finder</code>). Residual bias risk: the branch-keyword mapping in <code>data.js</code> reflects the author’s own judgement of which interests map to which branch and was not validated against a counsellor or larger dataset.
Reliability	Offline mode guarantees the app functions with zero external dependency; Live mode has explicit error handling that surfaces API failures in the UI rather than failing silently.
Ethical AI usage	System prompts explicitly instruct the assistant never to fabricate specific college/exam/scholarship facts and to say so when uncertain; salary/outlook figures are always presented as ranges with caveats, never guarantees; sensitive personal fields are clearly optional and scoped.

8 Minimizing Hallucinations

Strategies actually implemented in the running prompts (not just discussed in the abstract):

- **Closed reference lists.** College Comparison, Exam Recommendation, and Scholarship Matching prompts each embed the full candidate dataset in the prompt and explicitly instruct: “*do not add colleges/exams/schemes outside this list.*”
- **Forced structured output + refusal path.** Every schema includes `needs_clarification` and

PathFinder
GenAI Career & College Admission Assistant

Offline demo mode

1. Profile
2. Assistant
3. Colleges
4. Prompt Lab
5. Saved & History
6. Settings

Guided Assistant

Profile Intake & Clarification

Technique: Role Prompting + Structured Prompting (ambiguity handling)

OFFLINE SIM

Profile Complete
☒ Yes

Missing Or Ambiguous
None

Clarifying Questions
None

Reasoning
All required fields are present; proceeding to career assessment.

► View prompt used (v2)

Career Assessment & Branch Recommendation

Technique: Role Prompting + Structured Prompting + Few-Shot Prompting

OFFLINE SIM

Needs Clarification
☒ No

Questions
None

Recommendations

- Branch**
CSE
FitScore
37
Reasoning
Computer Science & Engineering; matches stated interest(s) "coding"; aligns with strongest subject (Maths).
- Branch**
AIDS
FitScore
37
Reasoning
AI & Data Science; matches stated interest(s) "data science"; aligns with strongest subject (Maths).
- Branch**
EEE
FitScore
15
Reasoning
Electrical & Electronics Engineering; aligns with strongest subject (Maths).
- Branch**
ECE
FitScore
10
Reasoning
Electronics & Communication Engineering; weak signal from current profile — worth exploring if other options don't fit.
- Branch**
MECH
FitScore
10
Reasoning
Mechanical Engineering; weak signal from current profile — worth exploring if other options don't fit.

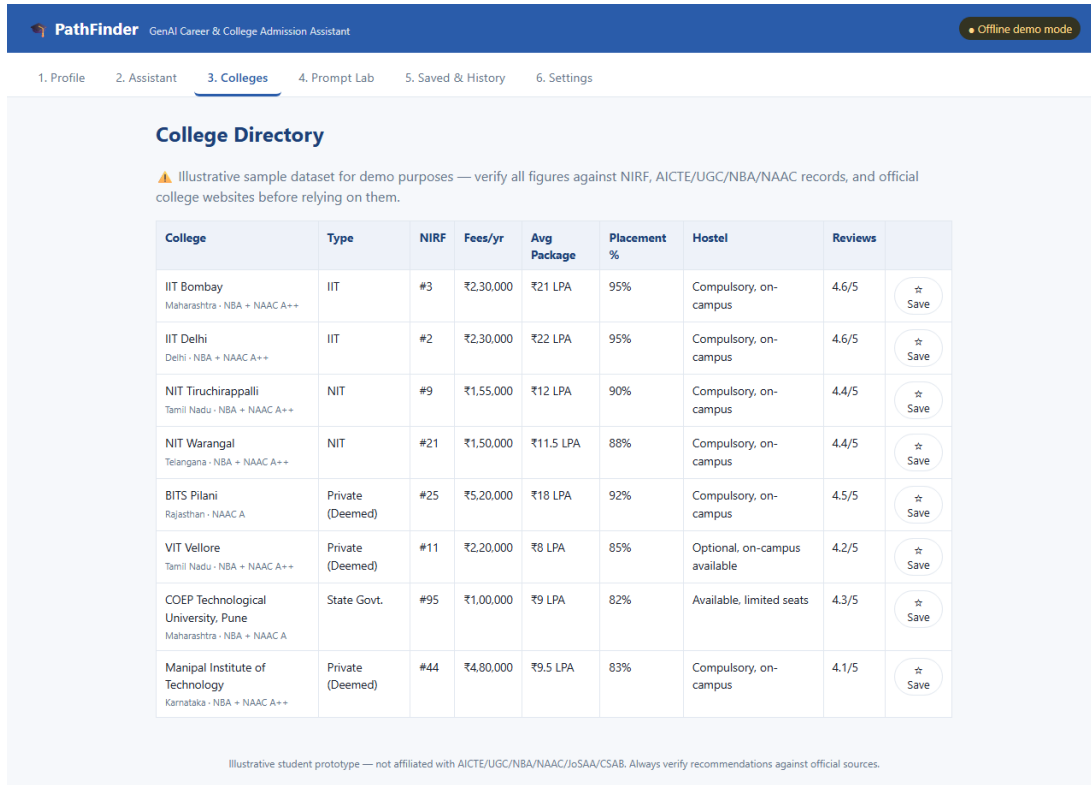
Pick your target branch to continue:

☒ CSE (37)
☐ AIDS (37)
☐ EEE (15)
☐ ECE (10)
☐ MECH (10)

► View prompt used (v2)

Illustrative student prototype — not affiliated with AICTE/UGC/NBA/NAAC/JoSAA/CSAB. Always verify recommendations against official sources.

Figure 2: Assistant tab – Career Assessment stage output with reasoning, branch picker, and prompt-view toggle.



College	Type	NIRF	Fees/yr	Avg Package	Placement %	Hostel	Reviews	
IIT Bombay Maharashtra · NBA · NAAC A++	IIT	#3	₹2.30.000	₹21 LPA	95%	Compulsory, on-campus	4.6/5	☆ Save
IIT Delhi Delhi · NBA · NAAC A++	IIT	#2	₹2.30.000	₹22 LPA	95%	Compulsory, on-campus	4.6/5	☆ Save
NIT Tiruchirappalli Tamil Nadu · NBA · NAAC A++	NIT	#9	₹1.55.000	₹12 LPA	90%	Compulsory, on-campus	4.4/5	☆ Save
NIT Warangal Telangana · NBA · NAAC A++	NIT	#21	₹1.50.000	₹11.5 LPA	88%	Compulsory, on-campus	4.4/5	☆ Save
BITS Pilani Rajasthan · NAAC A	Private (Deemed)	#25	₹5.20.000	₹18 LPA	92%	Compulsory, on-campus	4.5/5	☆ Save
VIT Vellore Tamil Nadu · NBA · NAAC A++	Private (Deemed)	#11	₹2.20.000	₹8 LPA	85%	Optional, on-campus available	4.2/5	☆ Save
COEP Technological University, Pune Maharashtra · NBA · NAAC A	State Govt.	#95	₹1.00.000	₹9 LPA	82%	Available, limited seats	4.3/5	☆ Save
Manipal Institute of Technology Karnataka · NBA · NAAC A++	Private (Deemed)	#44	₹4.80.000	₹9.5 LPA	83%	Compulsory, on-campus	4.1/5	☆ Save

Illustrative student prototype — not affiliated with AICTE/UGC/NBA/NAAC/JoSAA/CSAB. Always verify recommendations against official sources.

Figure 3: College Directory tab with the illustrative-data disclaimer and save-to-shortlist controls.

`missing_or_ambiguous` fields so the model has an explicit, in-schema way to say “I don’t have enough information” instead of confabulating.

- **Verification hand-off fields.** College Comparison output includes `verify_before_deciding` (e.g., “current NIRF ranking”, “latest official placement report”); Counselling Guidance output includes `official_sources`; the system prompt for Counselling Guidance explicitly forbids stating specific current-year cutoff numbers.
- **Range, not point, claims.** Career Prospects always returns a `salary_range_lpa` plus a `caveats` array, never a single figure presented as certain.
- **Not yet implemented (future work):** `temperature=0` for factual stages in Live mode, and a retrieval step against a real, regularly-refreshed NIRF/AICTE dataset instead of the static illustrative one.

9 Protecting Student Data Privacy

- All profile data, shortlist, and history persist only in the browser’s `localStorage` – there is no backend and no server-side database in this prototype, so there is nothing to breach server-side.
- The Anthropic API key (Live mode) is likewise stored only in `localStorage` and is sent only to `api.anthropic.com` at the moment a stage is run – never to any third party, never logged.
- Sensitive optional fields (gender, category, income band, disability) are visually separated in the Profile form under a collapsed “optional, for scholarship matching only” section, and are structurally

PathFinder
GenAI Career & College Admission Assistant

Offline demo mode

1. Profile
2. Assistant
3. Colleges
4. Prompt Lab
5. Saved & History
6. Settings

Prompt Lab

Compare a minimal zero-shot prompt (V1) against the engineered prompt (V2 — role + structured schema + few-shot + template). Edit either template and re-run.

Stage **Career Assessment & Branch Recommendation**

V1 — Baseline (zero-shot)

```
Given this student profile, suggest 3 good engineering branches:
{"name":"","academicPercentage":88,"strongSubject":"Maths",
"interests":["coding","data science"],"examIntent":true,"collegeTypes":
["NII"],"preferredStates":["Maharashtra","Tamil Nadu"],
"budgetPerYear":200000,"currentMonth":"Aug 2026","disability":false,"branch":"CSE"}
```

Run V1

V2 — Engineered

```
## Example (few-shot)
Input profile: {"interests": ["circuits", "coding"],
"strongSubject": "Physics", "percentage": 88}
Good output:
{
  "recommendations": [
    {"branch": "ECE", "fitScore": 88, "reasoning":
"Circuits interest plus strong Physics directly matches
ECE's core coursework (signals, devices)."},
    {"branch": "CSE", "fitScore": 80, "reasoning":
"Coding interest is a strong CSE signal even though it's
secondary to circuits here."}
  ],
  "needs_clarification": false
}
```

Run V2

OFFLINE

Needs Clarification

✗ No

Questions

None

Recommendations

- Branch

CSE

FitScore

37

Reasoning

Computer Science & Engineering: matches stated interest(s) "coding"; aligns with strongest subject (Maths).
- Branch

AIDS

FitScore

37

Reasoning

AI & Data Science: matches stated interest(s) "data science"; aligns with strongest subject (Maths).
- Branch

EEE

FitScore

15

Reasoning

Electrical & Electronics Engineering: aligns with strongest subject (Maths).
- Branch

ECE

FitScore

10

Reasoning

Electronics & Communication Engineering: weak signal from current profile — worth exploring if other options don't fit.
- Branch

MECH

FitScore

10

Reasoning

Mechanical Engineering: weak signal from current profile — worth exploring if other options don't fit.

Illustrative student prototype — not affiliated with AICTE/UGC/NBA/NAAC/JoSAA/CSAB. Always verify recommendations against official sources.

Figure 4: Prompt Lab – V1 (zero-shot) and V2 (engineered) templates, with V2’s output rendered after running.

excluded from every other stage’s context (§7, Fairness row).

- A “Clear all data” control in Settings wipes everything from the browser in one action.
- **Production gap, stated honestly:** a real deployment should not ask each student to supply their own API key. It needs a server-side proxy holding the key, per-user authentication, encryption at rest for any persisted profile data, a documented data-retention/deletion policy, and compliance review (India’s DPDP Act, 2023, for a system handling minors’ data) – none of which a client-only prototype can or should attempt to fake.

10 Validation Against Official Sources

Because this environment has no access to live AICTE / UGC / NBA / NAAC / NIRF / JoSAA / CSAB feeds, the dataset in `data.js` is explicitly labelled illustrative rather than presented as verified. The validation workflow a real deployment must run before trusting any figure shown to a student:

1. **Institutional accreditation & approval** – cross-check every listed college against the current AICTE-approved institutions list and UGC/NBA/NAAC accreditation records before display.
2. **Rankings** – refresh NIRF Engineering rankings from the official NIRF publication each cycle; timestamp every ranking shown with a “last verified on” date.
3. **Counselling data** – seat matrices, cutoffs, and round dates must be pulled from JoSAA/CSAB or the relevant state admission committee at the start of each admission cycle; the app’s Counselling Guidance stage is deliberately designed to explain *process*, never state specific cutoff numbers, for exactly this reason.
4. **Fees, placements, and reviews** – sourced from official college websites and their published, audited placement reports, not third-party aggregators, with a visible source citation per figure.
5. **Recurring re-validation** – given yearly churn in cutoffs, fees, and placement stats, any production version needs an annual data-refresh pipeline, not a one-time import.

11 Limitations & Future Work

- The offline simulator demonstrates the prompt design’s intended behaviour deterministically but is not itself a language model; genuine reasoning quality, robustness to unusual phrasing, and multi-turn conversational nuance can only be assessed in Live mode with a real API key.
- The reference dataset (8 colleges, 7 exams, 6 scholarship schemes) is a small illustrative sample, not the full national landscape, and is not currently re-validated on any schedule.
- No authentication, multi-device sync, or server persistence exists; the app is single-browser, single-device by design for this prototype.
- Only one model provider (Anthropic Claude) is integrated; a production system may want provider redundancy.
- Free-text refine/feedback input has no moderation layer – acceptable for a course prototype, not for a public deployment.

- Accessibility (screen-reader labelling, keyboard navigation audit) and multi-language support (most target students are more comfortable in a regional language for this decision) were out of scope here but matter for real-world reach and fairness.

12 Conclusion

PathFinder demonstrates the full guidance journey the assignment specifies – profile intake with ambiguity handling, chained multi-stage recommendations, multi-parameter college comparison, exam and scholarship matching, roadmap generation, and counselling explanation – built on a documented library of role, structured, template, few-shot, chaining, and iterative-refinement prompts, with every recommendation traceable to the exact prompt that produced it. Its most important design choice is refusing to pretend: illustrative data is labelled as such, salary/placement claims are ranges with caveats, and the evaluation and validation sections above report real gaps rather than assumed compliance – in keeping with the assignment’s own emphasis on accuracy, transparency, and ethical AI usage over polish.